

FPS-3000 — VersaBus card trace worksheet

For tracing the **V-BUS XLTR (612-4803-400-G)** and **AP I/F (612-4448-401-F)** cards with a continuity tester. Every expectation below is a prediction derived from the SBC firmware, not an observation of the board. Where a check disagrees with the prediction, the board wins and the prediction is wrong — record what you find.

Source: refs_extracted/versabus_access_map.md. Pin numbers for the BIM are from the Motorola MC68153 datasheet; VersaBus P1 pin numbers from refs_extracted/versabus_pinout.md.

What the firmware says is on the XLTR card

The SBC writes a block at \$FF0230-\$FF025F in a pattern that matches three four-channel Bus Interface Modules, 16 bytes apart. Ten of the registers receive interrupt vector numbers \$41-\$4A, and each of those numbers resolves to a distinct interrupt handler in the firmware. The card list calls this card "V-BUS XLTR 3 BIMS".

Check 1 — identify the three BIM packages

Look for three 40-pin DIPs. Read and record the part number on each. MC68153 is the expected part; anything with the same pinout is equally consistent.

	Package ref (e.g. U12)	Part number as marked	Date code	Pin 1 orientation OK?

Confirm each candidate is a BIM before trusting it: pin 3 = CS, pins 38/39/40 = A1/A2/A3, pins 28,29,32-37 = D0-D7, pin 18 = CLK, pin 4 = DTACK.

Check 2 — the interrupt level (highest value)

This check settles an open question. The firmware writes \$5F to the control registers of five channels and \$5E to one. Read against the MC68153 datasheet, bits 0-2 of the control register choose which IRQ output the chip asserts, so \$5F selects IRQ7 and \$5E selects IRQ6. Our emulator assumed level 5 until this analysis. Buzz the IRQ pins to the VersaBus P1 interrupt lines and the question closes.

BIM IRQ pin	Signal	VersaBus P1 pin	Connected? (Y/N)	Notes
8	IRQ1*	87		
12	IRQ2*	88		
13	IRQ3*	89		
14	IRQ4*	90		
15	IRQ5*	91		
16	IRQ6*	92		
17	IRQ7*	93		

Prediction: pin 17 (IRQ7*) is wired on the BIMs at \$FF0240 and \$FF0250. Pin 16 (IRQ6*) is wired on the BIM at \$FF0230. If instead IRQ5* (pin 15, P1 pin 91) is the one connected, our datasheet reading is wrong and the old level-5 assumption was right. Either result is useful; record which pins actually go anywhere.

Check 3 — the IACK daisy chain

All five enabled channels request the same level, so the hardware separates them with the IACKIN*/IACKOUT* chain. Establish the order the three BIMs sit in.

From	Pin	To	Pin	Connected?
BIM ___ IACKOUT*	7	BIM ___ IACKIN*	6	
BIM ___ IACKOUT*	7	BIM ___ IACKIN*	6	
VersaBus IACKIN		first BIM IACKIN*	6	
last BIM IACKOUT*	7	VersaBus / next card		

Chain order determines which channel wins when two interrupt at once. Record the physical order left-to-right on the card as well as the electrical order.

Check 4 — address decode and chip select

Each BIM answers a 16-byte window. A1-A3 select the register inside it, so the window is chosen by decoding A4 and above into the CS pin.

BIM	Window	Registers	A6	A5	A4	Which package has this CS?
BIM0	\$FF0230-\$FF023F	CR0-3 then VR0-3	0	1	1	
BIM1	\$FF0240-\$FF024F	CR0-3 then VR0-3	1	0	0	
BIM2	\$FF0250-\$FF025F	CR0-3 then VR0-3	1	0	1	
(XLTR ctrl)	\$FF0200-\$FF021B	mode/status file	0	0	0	

A4 and A5 alone do not separate the windows: BIM1 and the XLTR control file both read A4=0, A5=0. A6 is the bit that tells them apart, so the decoder must use at least A4, A5 and A6.

Trace pin 3 (CS) of each package back to whatever generates it (PAL, decoder, gate). Note the part number of the decoder. That part tells us how much of the \$FF0200 window is decoded and whether the gaps at \$FF021C-\$FF022F are real.

Package	CS source (part + designator)	Which address lines feed it	Notes

Check 5 — register layout confirmation

The layout below is inferred from firmware writes. It predicts that CR n and VR n are the same channel. If the board disagrees, the whole channel-to-task mapping in the access map is wrong.

Offset	A3	A2	A1	Register	Channel	Firmware writes
+\$0	0	0	0	CR0	ch0	\$5E or \$5F or \$00
+\$2	0	0	1	CR1	ch1	\$00
+\$4	0	1	0	CR2	ch2	\$5F
+\$6	0	1	1	CR3	ch3	\$5F or \$00
+\$8	1	0	0	VR0	ch0	vector number
+\$A	1	0	1	VR1	ch1	vector number
+\$C	1	1	0	VR2	ch2	vector number
+\$E	1	1	1	VR3	ch3	vector number

Note the address arrives on the VersaBus as A01-A03 and reaches the BIM as A1-A3 (pins 38, 39, 40). Confirm there is no swap or inversion in between: a reversed A1/A3 would exchange the CR and VR halves and everything above would still look self-consistent.

Check 6 — who drives the device interrupt inputs

The four INT inputs on each BIM are what the rest of the chassis pulls to request service. Tracing these backwards tells us which chassis function owns each firmware task.

BIM	INT pin	Datasheet name	Firmware owner (predicted)	What actually drives it
BIM1	23	INT2	TCBXP1I (XP-32 channel 1)	
BIM1	24	INT3	TCBXP2I/3I	
BIM2	19	INT0	TCBXP3I/2I	
BIM2	22	INT1	TCBXP4I	
BIM2	23	INT2	TCBIO1I (host link)	
BIM0	19	INT0	unidentified, vector \$41	

The last row matters most. Vector \$41 reaches handler F04930 and we do not know what it serves. Whatever drives BIM0 INT0 answers that.

Check 7 — AP I/F channel windows

On the AP I/F card (612-4448-401-F), the firmware uses four channel windows on a 32-byte stride. Each window has one write port and three read ports. The card carries eight Am29705 16x4 dual-port SRAMs, which give 32 bits of width.

Offset	Dir	ch1	ch2	ch3	ch4	Role
+\$04	W	FFF0044	FFF0064	FFF0084	FFF00A4	write port
+\$08	R	FFF0048	FFF0068	FFF0088	FFF00A8	read port A (consumes a host byte)
+\$0A	R	FFF004A	FFF006A	FFF008A	FFF00AA	status, host presents \$4F
+\$0E	R	FFF004E	FFF006E	FFF008E	FFF00AE	read port B

Useful checks here: do the four windows share one set of Am29705s with the channel number selecting a bank, or is there separate silicon per channel? And which pins of the two ribbon connectors carry the host side? The host-side counterpart card is missing, so its pinout has to come off this card.

Question	Finding
Am29705 designators (expect 8)	
Do all four channels share them?	
Ribbon connector 1: pin count / signals	
Ribbon connector 2: pin count / signals	
What generates the \$4F status value?	

Check 8 — the two Am29116 on the XP32 EXEC card

The EXEC card (612-4805-002-R) carries **two** Am29116 devices. Hockney describes one executive unit running from a 2K x 80-bit PROM, which leaves open how the pair divide the work. Settling this changes how we read a PROM dump, so it is worth doing before the PROMs come off.

Question	Why it matters	Finding
Do both share one instruction field, or does each have its own?	Decides whether to buffer the 800s	Each has its own instruction stream or two
Is there a carry or status link between them (carry chain)?	Carry chain or link?	As one 32-bit ALU
Do they share the same clock and PROM address bus?	Simple and they sequence the sequence of programs	
Which PROM outputs reach which Am29116 inst?	Gives the 12-lane assignment directly, no search needed	

That last row is the highest-value trace on the whole card. The PROM dump arrives as N separate chips and we will not know which chip supplies which bits of the 80-bit word. Tracing PROM data pins to Am29116 instruction pins hands us that mapping and removes a search over chip orderings.

PROM inventory (fill in before removing anything)

[illegible]

Photograph the card with designators legible before pulling any device. Once the PROMs are out of their sockets, the physical order is the one thing that cannot be recovered from the dumps.

Notes

